Quiz 8: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 8 is Thursday Oct 22, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Scope

The Oct 15 lecture, which was the sequence models deck, plus Lab 7. That meeting was the second half of the day; the first half finished object detection and segmentation, and **none of that is on this quiz.** Detection and segmentation were Quiz 7's subject and they stay there.

On the quiz:

- what makes data sequential, and which two assumptions convolution makes that sequences break
- tokenization at character, word and subword level, and the vocabulary
- one-hot encoding, and why an embedding replaces it
- padding a ragged batch
- masking, in all three places it is needed
- `collate_fn` and bucketing by length
- 1D convolution over a sequence, and its receptive field
- the recurrence, weight sharing across time, and reading an RNN's output
- autoregressive generation and teacher forcing
- backpropagation through time and the two failures it produces
- what gating fixed and what it left alone
- everything you did in Lab 7

Not on the quiz. Anything from the transformer, which is the Oct 22 lecture and therefore Quiz 9: self-attention, positional encoding, multi-head attention, the encoder-decoder. Also off: GRU internals, bidirectional RNNs, and writing an LSTM cell from scratch. The lecture's "What carries forward" slide set those aside on purpose and said you would not be asked to build one.

The register for this quiz is the same as the lecture's. You should be able to say **what each piece is for** and **why the field left recurrence behind**. The only arithmetic is the arithmetic you already did in the lab: padding shares, the pooling error, receptive fields, and reading shapes.

Part 1: What makes data sequential

Sequence data is data where the order of the elements carries meaning and the number of elements is not fixed. Both halves matter.

Images care about order too, since a bag of pixels in random order is not an image. The difference is size: an image's size is fixed by resizing it to 224×224 before it goes in, and **you cannot resize a sentence**.

A convolution assumes two things, and a sequence breaks both:

convolution assumes	sequences
a fixed-size input	length varies per example
the neighbours are the context	the context can be 60 tokens away

The second one is the deeper problem, and it is worth being precise about why. `nn.Conv1d` accepts any length `T` with the same weights, so **variable length was never the real objection** to using convolution on text. Locality is.

Part 2: Tokens and the vocabulary

Tokenization splits text into pieces. A vocabulary is the lookup from those pieces to integer ids. The pieces can be characters, words or subwords, and the choice trades sequence length against vocabulary size.

The three granularities

level	sequence length	vocabulary	unseen input
character	longest	~65 symbols	spells anything
word	shortest	never stops growing	becomes `<unk>`
subword	between	fixed	spells anything

The lecture's sentence came out as 37 character tokens, 8 word tokens and 11 subword tokens. Work grows with sequence length, which is the cost of characters. The cost of words is the other column.

Why a word vocabulary cannot be made big enough

Lab 7 measured this on 187,000 word tokens. Distinct lowercase words seen after the first part of the corpus:

fraction of corpus	distinct words
5%	2,961
10%	4,788
25%	9,165
50%	14,665
100%	22,934

Going from half the corpus to all of it added **8,269 new words**. The curve is still climbing when the data runs out, and it would keep climbing on more, because names, numbers, typos and inflections never run out.

What capping it costs

Keep the k most frequent words, map the rest to `<unk>`. The share of **tokens** destroyed, measured in Lab 7:

cap	tokens that become `<unk>`
1,000	27.40%
2,000	20.98%
5,000	12.78%
10,000	6.92%
20,000	1.57%

A 10,000 word vocabulary is not small, and it still throws away one token in fifteen. Everything thrown away becomes the same symbol, so the model cannot tell a proper noun from a typo.

Why subwords won

Byte pair encoding starts from characters and repeatedly merges the most frequent adjacent pair until it reaches the vocabulary size you asked for. Frequent words survive whole; rare ones split into pieces that are themselves in the vocabulary.

In Lab 7, BPE with a vocabulary of **500** produced **0.0000%** `<unk>` over 2,000 sentences. Compare that with 6.92% from a word vocabulary twenty times larger. `resolved` can always fall back to `res` + `ol` + `ved`, so nothing is unspellable.

> **Likely question.** Why does a subword vocabulary of 500 have no `<unk>` when a word vocabulary of 10,000 has 6.9%? Because any word can be spelled from pieces that are in the vocabulary.

Reserved ids

By convention `<pad>` is 0 and `<unk>` is 1, and real tokens start at 2. This is not decoration. If you build `{w: i for i, w in enumerate(vocab)}` then your most frequent word gets id 0, which is also the pad id, and from that moment padding and that word are the same thing. Your mask deletes it everywhere.

Part 3: One-hot, then embeddings

A one-hot vector is all zeros with a single 1, at the index of the token. A sequence of s tokens over a vocabulary of n is an $s \times n$ matrix $\mathbf{O}$, almost entirely zeros.

Multiply it by a weight matrix and you get something dense:

$$\mathbf{E} = \mathbf{O}\,\mathbf{W_E} \qquad (s \times n)(n \times m) = (s \times m)$$

with m in the hundreds rather than the tens of thousands.

But multiplying a one-hot row by $\mathbf{W_E}$ just selects a row of $\mathbf{W_E}$, so there is no reason to do the multiply. `nn.Embedding` indexes instead. Lab 7 printed the identity directly:

```
emb(tensor(7)) == emb.weight[7]  ->  True
```

> **Likely question.** Does an embedding layer perform a matrix multiply? No. It is a lookup table of learned vectors, one row per token, and it indexes.

The rows are parameters, so the model learns them, and tokens that behave alike end up near each other. One-hot can never do that: every token sits exactly as far from every other as it started, forever.

Shapes and counts

`nn.Embedding(V, d)` maps $(N, T) \to (N, T, d)$ and holds $V \times d$ parameters. So `nn.Embedding(5000, 128)` on a $(16, 30)$ batch gives $(16, 30, 128)$ and holds 640,000 parameters. **Be able to do this both ways.**

`padding_idx`

`nn.Embedding(V, d, padding_idx=0)` does two things: it initializes row 0 to zeros, and it blocks gradient from ever reaching that row. So the pad token embeds to the zero vector permanently.

That is worth knowing for its own sake, and it is also what makes the next section's result exact.

Part 4: Padding

A batch has to be a rectangle, because a tensor has a shape. Padding appends the reserved id to every sequence until they all reach the length of the longest one.

For a batch of lengths $\ell_1 \dots \ell_N$:

$$T = \max_i \ell_i \qquad \text{cells} = N \times T \qquad \text{padding share} = 1 - \frac{\sum_i \ell_i}{N \times T}$$

Worked example, which is the kind of thing Part C asks. Lengths 3, 7, 12, 2:

- $T = 12$
- cells $= 4 \times 12 = 48$
- real tokens $= 3 + 7 + 12 + 2 = 24$
- padding $= 24/48 = 50\%$

T comes from this batch, not from the dataset. It changes every batch. A script that pads to a fixed global length has a bug, and that is what `collate_fn` exists to prevent.

Lab 7's random batch of 8 real sentences was 52.1% padding. More than half of that tensor meant "nothing here".

Part 5: Masking

A mask is a boolean tensor, the same shape as the batch, marking which positions are real. It is `ids != PAD`. Without one, every layer downstream reads `PAD` as a word.

The exact error, and why it is exact

Because `padding_idx` pins the pad vector to zero, the unmasked mean is the sum of the real vectors divided by T, while the correct mean divides by ℓ:

$$\text{naive} = \frac{v}{T}, \qquad \text{masked} = \frac{v}{\ell} \qquad \Rightarrow \qquad \text{naive} = \frac{\ell}{T}\,\text{masked}$$

so the relative error is **exactly** $1 - \ell/T$. Lab 7 measured this and the measured column matched the formula column to the digit for all eight rows, with a correlation between length and error of exactly -1.

length	T	error
2	24	91.7%
12	24	50.0%
24	24	0.0%

Likely question. A batch is padded to $T = 20$, one sentence is 5 tokens, and somebody pooled with `.mean(dim=1)`. How wrong is that sentence's vector? $1 - 5/20 = 75\%$.

Why it hides

Read that table again. **A masking bug damages short examples in proportion to how short they are, and leaves the longest member of each batch untouched.** The long examples are correct even with the bug, so the average over a validation set looks only mildly wrong. That asymmetry is the thing to recognize: wild behaviour on short inputs and clean behaviour on long ones points at masking before it points at the model.

The three places

where	what it looks like
pooling	divide by the real count, not T
the loss	`ignore_index=0`
attention, next week	pad positions go to $-\infty$ before the softmax

Forgetting any one of them is silent. Nothing raises.

On the loss specifically, Lab 7 compared two hypothetical models on a batch that was 52.1% padding:

model	unmasked loss	with `ignore_index`
gets every `PAD` right, every word wrong	**4.306**	8.987
gets every word right, every `PAD` wrong	4.681	**0.000**

Under the unmasked loss the useless model scores better, so gradient descent walks towards it. That follows from more than half the target cells being `PAD`, so more than half the available credit is for predicting "nothing here".

Part 6: Building the batch

A `Dataset` returns one example. The default collation stacks examples, which needs matching shapes, and ragged sequences do not have them, so it raises.

`collate_fn` receives the list of examples and returns whatever the model wants. For ragged data: pad to the longest member of **this** batch and hand back the mask alongside the ids, so nothing downstream has to guess.

Bucketing

T is set by the longest member of each batch, so sorting by length first makes batches internally uniform and most of the padding disappears. Lab 7 measured it on 2,048 sentences:

batch size	shuffled	sorted
8	60.2% padding	2.2%
32	73.5% padding	10.1%

You would not sort the whole dataset in practice, because then every epoch sees the same batches. The usual compromise is to shuffle, take a pool of a few hundred, sort inside the pool, and batch from that.

The one place order is not yours to change

Bucketing reorders sentences, which is harmless because they are independent. **A time series is not independent.** Split it on time: train on the past, validate on the future. A random split lets the model see tomorrow while it learns yesterday, and the validation number it reports then means nothing.

Part 7: 1D convolution

Slide a kernel along the sequence instead of across an image. `nn.Conv1d` takes (N, C, T), note the order, and accepts any T.

A kernel of width k widens the receptive field by $k-1$ per layer, so L stacked layers see $L(k-1)+1$ positions. With $k=3$ that is $2L+1$.

> **Likely question.** How many layers of kernel 3 before one position can see a token 100 away? You need 101 positions in view, so $2L+1 \geq 101$ and $L \geq 50$. Attention needs one.

That gap is the whole argument. Convolution reaches a constant distance further per layer; attention reaches everywhere in one hop.

This is current practice, not history. Speech models put a convolutional front end before the transformer, Conformer blocks run convolution and attention side by side, and dilated causal convolutions are a strong and cheap baseline for time series. Use convolution where the pattern is local.

Part 8: The recurrence

A recurrent network keeps a hidden state h_t that it updates at every step from the current input and the previous state.

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)\,, \qquad h_0 = \mathbf{0}$$

The same weights run at every step. That is **weight sharing**, the same idea as a kernel sliding over an image, except over time, and it is what lets one model accept any length.

It is also the problem. The only route from the first token to the last is along the W_h arrows, one multiplication per step, **with no other path**.

Reading the output

`nn.RNN` and `nn.LSTM` return `(output, h_n)`. With `batch_first=True`, `output` is (N, T, H): the hidden state at every step. Which part you read is what makes it a different task.

task	what you read
classify the whole sequence	the last state
label every token	all T states
translate	all states, via attention, which is next week

With padding, `out[:, -1]` is the state after reading pad tokens, not after reading the sentence. You want the state at each sequence's own final real position. This is the masking problem wearing a different hat.

Generating one token at a time

Predict one token, append it to the input, run again. That is how every text generation model you have used produces output: one token per forward pass. **Attention does not change this.**

Training does not work that way. Feeding the model its own early predictions compounds the errors, so you feed it the true previous token instead. That is **teacher forcing**, and it is why training can be parallel over positions while generation cannot.

Part 9: Backpropagation through time, and the two failures

Unroll it. T steps become T copies of the same layer, an ordinary feedforward network T deep, and you backpropagate through that. Because the weights are shared, W_h collects a gradient from every step. The name **backpropagation through time** is the only new part.

Failure one: the gradient does not reach back

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

A product of $T - t$ terms. Smaller than 1 and it goes to zero; larger and it explodes. **No choice of weights makes a product of many terms behave like a sum.**

Exploding is the easy half, because you can clip the norm. Vanishing is the hard half, because there is nothing to clip. The lecture measured the vanilla RNN keeping **0.53** of the gradient per step, which gives a usable reach of about **20 steps**.

Week 6 hit this same wall with depth in CNNs, and there the fix was a residual connection: a path with gradient 1 running alongside the layers. **Recurrence has no such path, because the recurrence is the path.**

Say the cost precisely. It is not that the model is wrong about long sequences. **It is that it cannot learn anything that depends on one**, because the signal that would teach it never arrives. A dependency 60 tokens apart is easy to represent and unreachable by gradient descent. Training longer does not help, and nor does a bigger hidden state.

Failure two: one step at a time

h_t needs h_{t-1}, so the steps cannot run together. The cost is one sequential step per token no matter what hardware you own. The lecture measured an LSTM at **16.00 ms** against attention's **7.63 ms** on 1,024 tokens, and attention does *more* total arithmetic.

> **Likely question.** Attention does more work and still finishes sooner. Why? Because the LSTM's steps are sequential and attention's are not. **The win is parallelism, not less work.** Attention's total work grows with the square of the length, which is the price it pays for that.

Part 10: What gating fixed, and what it did not

An LSTM is a recurrent network with a second state that information travels along without being multiplied by a weight matrix at every step.

The cell state is updated almost additively:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad h_t = o_t \odot \tanh(c_t)$$

Each gate is a learned function of x_t and h_{t-1} squashed to $[0, 1]$, so each is a soft switch: what to forget, what to write, what to expose.

When f_t is near 1, $c_t \approx c_{t-1}$, so the gradient along that track is multiplied by roughly 1 instead of by a weight matrix. That is the residual connection's trick, arrived at twenty years earlier.

	fixed?
gradient reach	partly: 28 steps against 20, still exponential
one step per token	not at all

Gating bought the field a decade without changing either of the two things that actually needed changing. That is the honest summary, and it is why the lecture spent five minutes on it.

Part 11: What Part D will look like

Part D is a script that runs, prints plausible numbers, and is wrong. It has three bugs and you give the line number and the consequence for each.

Read the stated symptom first. It tells you where to look. A symptom that singles out short sequences is a padding or masking symptom.

Bugs worth being able to spot on sight:

- a vocabulary built without reserving 0, so a real word collides with the pad id
- `nn.Embedding` without `padding_idx`, so the pad row learns a vector
- padding to a fixed global length instead of the batch's longest member
- pooling with `.mean(dim=1)` when a mask was passed in and ignored
- a per-token loss without `ignore_index`
- `out[:, -1]` on a padded batch
- a random split on a time series

The pattern across all of these: **nothing raises.** You get a number, the number is wrong, and only the shape of the wrongness tells you.

How to prepare

1. **Re-run Lab 7 and read your own output.** Nearly every number on this quiz is one you already printed. The lab is the study guide with a kernel attached.
2. **Be able to do the padding arithmetic cold:** given four lengths, give T, the cell count and the padding share.
3. **Be able to apply** $1 - \ell/T$, and more importantly say what it implies about which examples a masking bug damages.
4. **Be able to say what each piece is for in one sentence:** the pad id, the mask, `collate_fn`, bucketing, `padding_idx`, the forget gate.
5. **Be able to name the two failures** and say which one gating helped.

The formula sheet carries the shapes, the padding formulas, the $1 - \ell/T$ identity, the receptive field, the recurrence, the product, and the LSTM update. It deliberately does not tell you what any of them are *for*. That part is the quiz.